VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Arun Kumar H(1BM24CS055)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Arun Kumar H(1BM24CS055), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

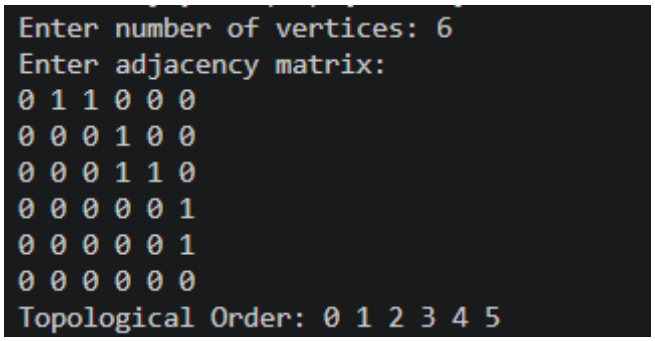
Lab program 1:Topological Sorting

```
#include <stdio.h>
#define MAX 10

int main() {
   int adj[MAX][MAX], indegree[MAX];
   int queue[MAX],  front  = 0, rear = 0;
   int n, i, j, count = 0, v;

   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         scanf("%d", &adj[i][j]);
      }
   }
   for(i = 0; i < n; i++) {
      indegree[i] = 0;
   }
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         if(adj[i][j] == 1)
            indegree[j]++;
      }
   }
```

```
    for(i = 0; i < n; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }
    printf("Topological Order: ");
    while(front < rear) {
        v = queue[front++];
        printf("%d ", v);
        count++;
        for(j = 0; j < n; j++) {
            if(adj[v][j] == 1) {
                indegree[j]--;
                if(indegree[j] == 0)
                    queue[rear++] = j;
            }
        }
    }
    if(count != n)
        printf("\nGraph contains a cycle");
    return 0;
}
```

OUTPUT:

Case 1:



Case 2:

```
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
1 0 0
0 0 1
Topological Order:
Graph contains a cycle
```

Case 3:

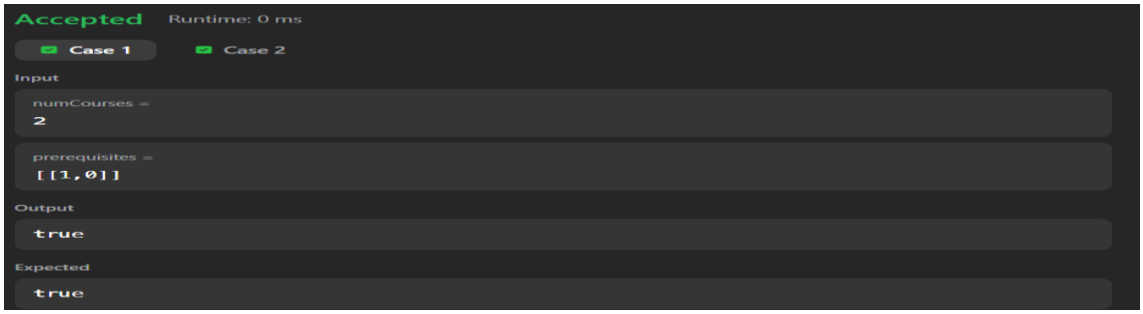
```
Enter number of vertices: 1
Enter adjacency matrix:
0
Topological Order: 0
```

Leetcode Problem(207.Course Schedule):

```
#include <stdbool.h>
#include <stdlib.h>
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int* size = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        size[b]++;
    }
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(size[i] * sizeof(int));
        size[i] = 0;
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][size[b]++] = a;
        indegree[a]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
```

```
    int count = 0;
    while (front < rear) {
        int node = queue[front++];
        count++;
        for (int i = 0; i < size[node]; i++) {
            int next = graph[node][i];
            indegree[next]--;
            if (indegree[next] == 0) {
                queue[rear++] = next;
            }
        }
    }
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(indegree);
    free(size);
    free(queue);
    return count == numCourses;
}
```

OUTPUT:


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true

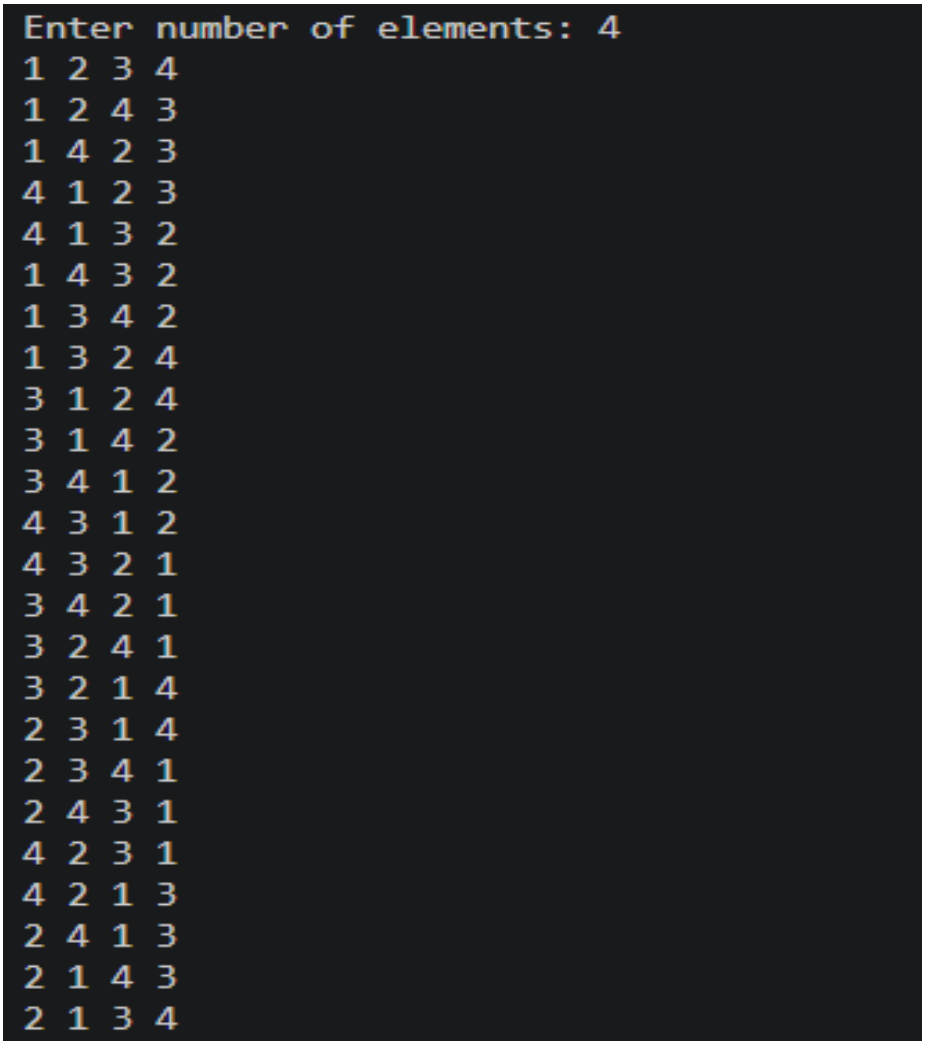

Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0],[0,1]]
Output
false
Expected
false

Lab program 2:Johnson Trotter Algorithm

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1
int getMobile(int a[], int dir[], int n) {
   int mobile = 0, mobileIndex = -1;
   for(int i = 0; i < n; i++) {
      if(dir[i] == LEFT && i != 0 && a[i] > a[i - 1] && a[i] > mobile) {
         mobile = a[i];
         mobileIndex = i;
      }
      if(dir[i] == RIGHT && i != n - 1 && a[i] > a[i + 1] && a[i] > mobile) {
         mobile = a[i];
         mobileIndex = i;
      }
   }
   return mobileIndex;
}
void printPermutation(int a[], int n) {
   for(int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}
int main() {
   int n;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   int a[n], dir[n];
```

```
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(a, n);
    while(1) {
        int mobileIndex = getMobile(a, dir, n);
        if(mobileIndex == -1)
            break;
        int swapIndex;
        if(dir[mobileIndex] == LEFT)
            swapIndex = mobileIndex - 1;
        else
            swapIndex = mobileIndex + 1;
        int temp = a[mobileIndex];
        a[mobileIndex] = a[swapIndex];
        a[swapIndex] = temp;
        temp = dir[mobileIndex];
        dir[mobileIndex] = dir[swapIndex];
        dir[swapIndex] = temp;
        mobileIndex = swapIndex;
        for(int i = 0; i < n; i++) {
            if(a[i] > a[mobileIndex])
                dir[i] = -dir[i];
        }
        printPermutation(a,  n);
    }
    return 0;
}
```

OUTPUT:


Enter number of elements: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

Lab program 3:Merge Sort

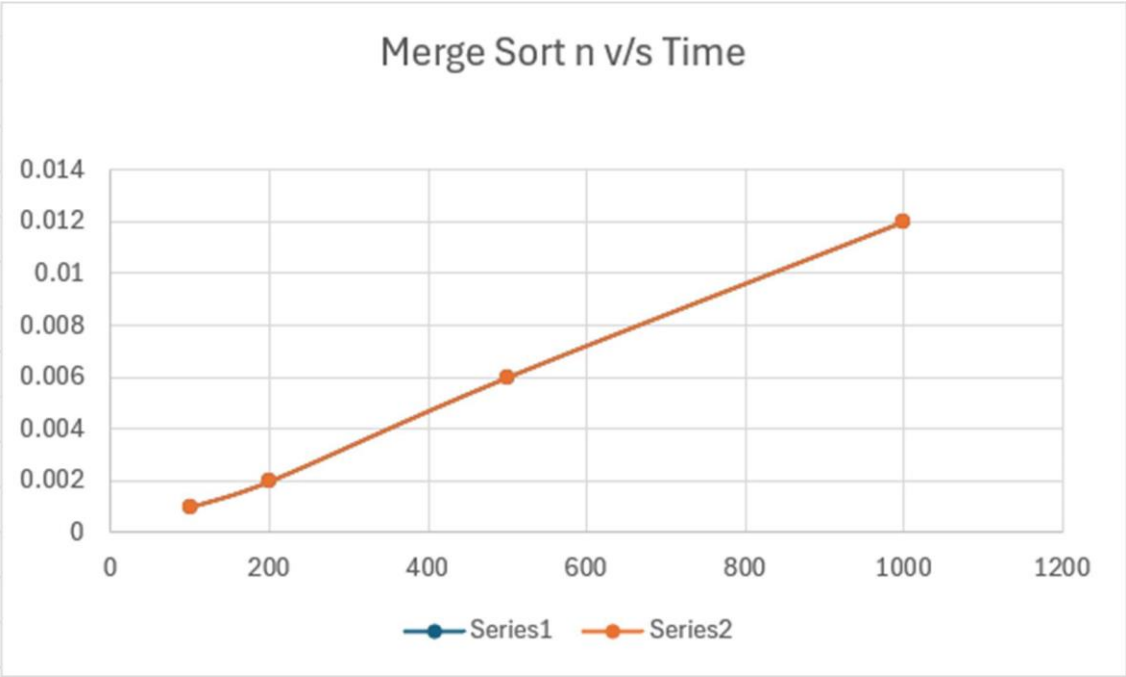
```
#include <stdio.h>
#include <time.h>
void merge(int a[], int low, int mid, int high) {
   int temp[100000];
   int i = low, j = mid + 1, k = 0;
   while(i <= mid && j <= high) {
      if(a[i] < a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }
   while(i <= mid)
      temp[k++] = a[i++];
   while(j <= high)
      temp[k++] = a[j++];
   for(i = low, k = 0; i <= high; i++, k++)
      a[i] = temp[k];
}
void mergeSort(int a[], int low, int high) {
   if(low < high) {
      int mid = (low + high) / 2;
      mergeSort(a, low, mid);
      mergeSort(a, mid + 1, high);
      merge(a, low, mid, high);
   }
}
int main() {
   int n, i, a[100000];
```

```
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    mergeSort(a, 0, n - 1);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements are:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements:
100
23
189
79
56
Sorted elements are:
23 56 79 100 189
Time taken = 0.000000 seconds
```

GRAPH:


Merge Sort n v/s Time
0.014
0.012
0.01
0.008
0.006
0.004
0.002
0
0
200
400
600
800
1000
1200
Series1
Series2

Lab program 4:Quick Sort

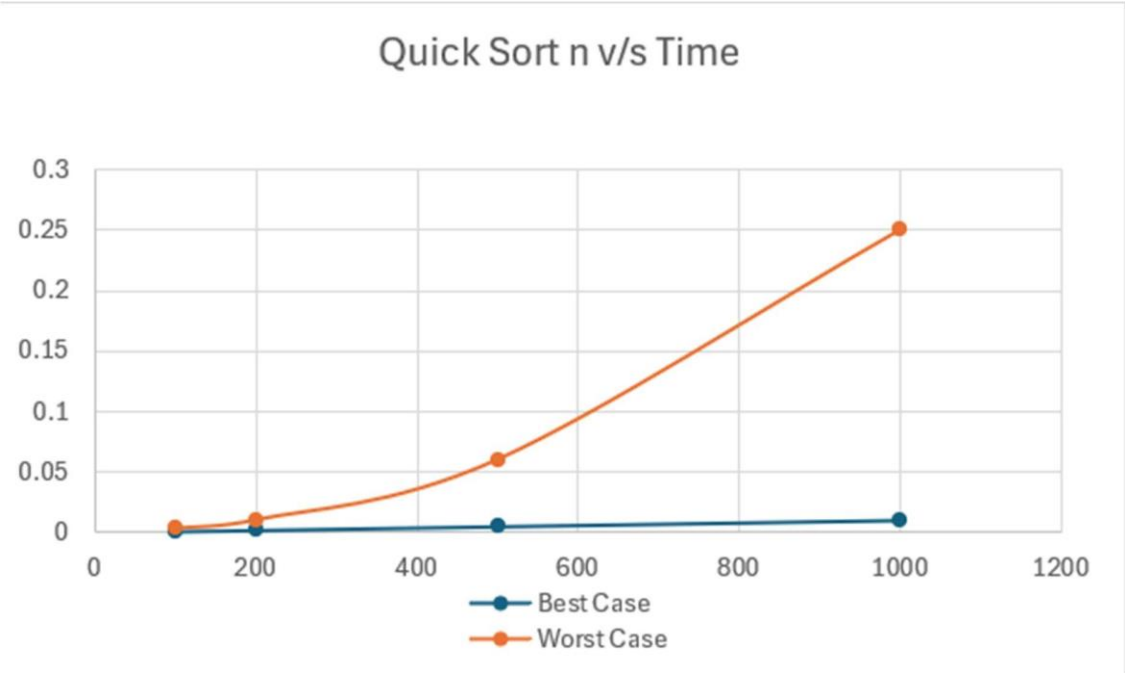
```
#include <stdio.h>
#include <time.h>
void quickSort(int a[], int low, int high) {
   int i, j, pivot, temp;
   if(low < high) {
      pivot = low;
      i = low;
      j = high;
      while(i < j) {
         while(a[i] <= a[pivot] && i < high)
            i++;
         while(a[j] > a[pivot])
            j--;
         if(i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }
      temp = a[pivot];
      a[pivot] = a[j];
      a[j] = temp;
      quickSort(a, low, j - 1);
      quickSort(a, j + 1, high);
   }
}
int main() {
   int n, i, a[100000];
```

```
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    quickSort(a, 0, n - 1);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements are:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT:

```
Enter no of elements: 8
Enter elements:
12 34 75 93 44 59 27 19
Sorted array:
12 19 27 34 44 59 75 93
Execution time = 0.000000 seconds
```

GRAPH:

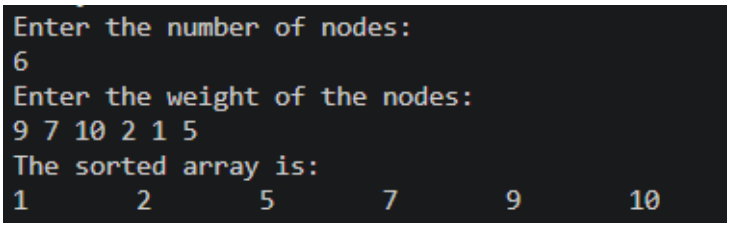

Quick Sort n v/s Time
0.3
0.25
0.2
0.15
0.1
0.05
0
0
200
400
600
800
1000
1200
Best Case
Worst Case

Lab program 5:Heap Sort

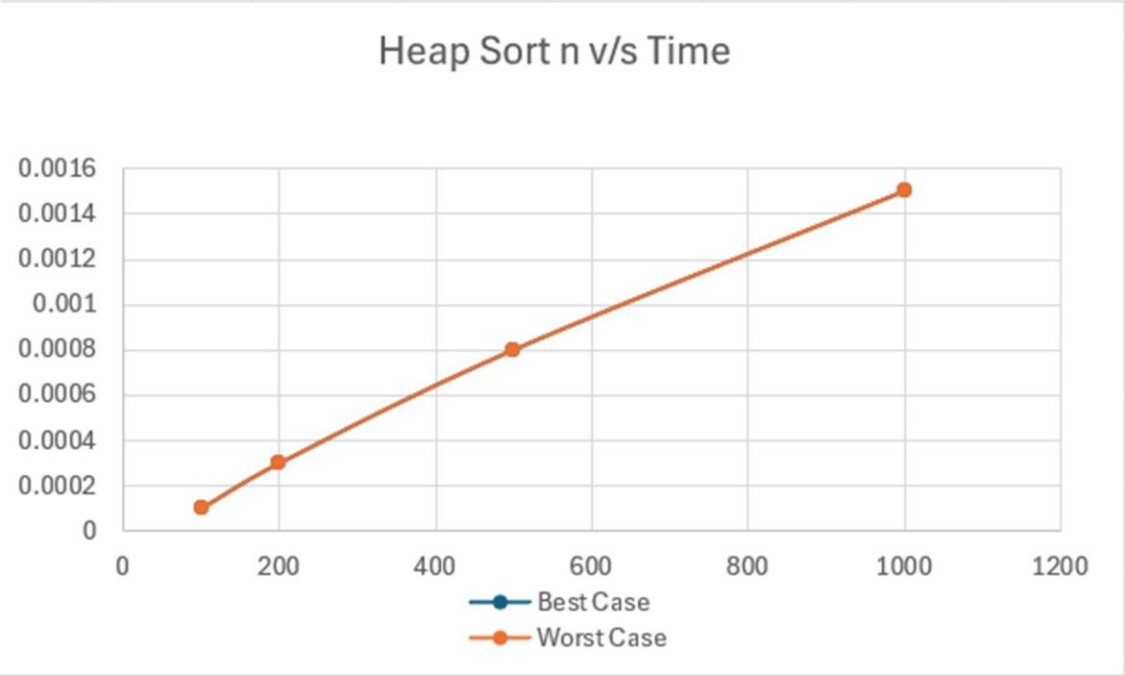
```
#include <stdio.h>
#include <time.h>
void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;
    if(left < n && a[left] > a[largest])
        largest = left;
    if(right < n && a[right] > a[largest])
        largest = right;
    if(largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n) {
    int temp;
    for(int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
    for(int i = n - 1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, i, 0);
    }
```

```
}
int main() {
    int n, i, a[100000];
    clock_t start, end;
    double time_taken;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    heapSort(a, n);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted elements are:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nTime taken = %f seconds\n", time_taken);
    return 0;
}
```

OUTPUT:



GRAPH:


Heap Sort n v/s Time
0.0016
0.0014
0.0012
0.001
0.0008
0.0006
0.0004
0.0002
0
0
200
400
600
800
1000
1200
Best Case
Worst Case

Lab program 6:0/1 Knapsack Problem Using Dynamic Programming

```
#include <stdio.h>
int max(int a, int b) {
    if(a > b)
        return a;
    else
        return b;
}
int knapsack(int n, int weight[], int profit[], int capacity) {
    int i, j;
    int dp[n + 1][capacity + 1];
    for(i = 0; i <= n; i++) {
        for(j = 0; j <= capacity; j++) {
            if(i == 0 || j == 0)
                dp[i][j] = 0;
            else if(weight[i] <= j)
                dp[i][j] = max(profit[i] + dp[i - 1][j - weight[i]],
                               dp[i - 1][j]);
            else
                dp[i][j] = dp[i - 1][j];
        }
    }
    return dp[n][capacity];
}
int main() {
    int n, i, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
```

```
    int weight[n + 1], profit[n + 1];
    printf("Enter weights:\n");
    for(i = 1; i <= n; i++)
        scanf("%d", &weight[i]);
    printf("Enter profits:\n");
    for(i = 1; i <= n; i++)
        scanf("%d", &profit[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    int result = knapsack(n, weight, profit, capacity);
    printf("Maximum profit = %d\n", result);
    return 0;
}
```

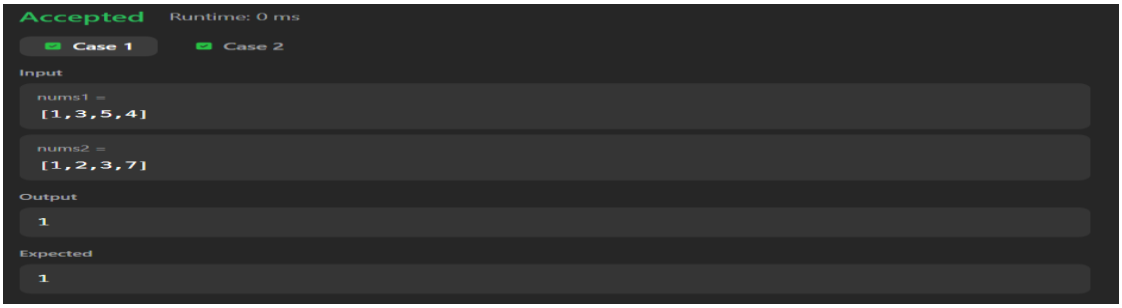
OUTPUT:

```
Enter number of objects: 5
Enter weights:
10 12 13 9 7
Enter profits:
10 20 16 12 14
Enter knapsack capacity: 24
Maximum Profit = 34
```

Leetcode Problem(801.Minimum Swaps):

```
#include <limits.h>
int min(int a, int b) {
    return a < b ? a : b;
}
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int newKeep = INT_MAX;
        int newSwap = INT_MAX;
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            newKeep = min(newKeep, keep);
            newSwap = min(newSwap, swap + 1);
        }
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            newKeep = min(newKeep, swap);
            newSwap = min(newSwap, keep + 1);
        }
        keep = newKeep;
        swap = newSwap;
    }
    return min(keep, swap);
}
```

OUTPUT:


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[1,3,5,4]
nums2 =
[1,2,3,7]
Output
1
Expected
1


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums1 =
[0,3,5,8,9]
nums2 =
[2,1,4,6,9]
Output
1
Expected
1

Lab program 7:Floyd's Algorithm

```
#include <stdio.h>
#define INF 999
void floyd(int n, int cost[10][10]) {
   int i, j, k;
   for(k = 0; k < n; k++) {
      for(i = 0; i < n; i++) {
         for(j = 0; j < n; j++) {
            if(cost[i][k] + cost[k][j] < cost[i][j])
               cost[i][j] = cost[i][k] + cost[k][j];
         }
      }
   }
   printf("All Pair Shortest Path Matrix:\n");
   for(i = 0; i < n; i++) {
      for(j = 0; j < n; j++) {
         if(cost[i][j] == INF)
            printf("INF ");
         else
            printf("%d ", cost[i][j]);
      }
      printf("\n");
   }
}
int main() {
   int n, i, j;
   int cost[10][10];
   printf("Enter number of vertices: ");
   scanf("%d", &n);
```

```
    printf("Enter cost matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
    floyd(n, cost);
    return 0;
}
```

OUTPUT:

Case 1:

```
Enter no. of vertices: 4
Enter adjacency matrix:
(Use 999 for INF)
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
```

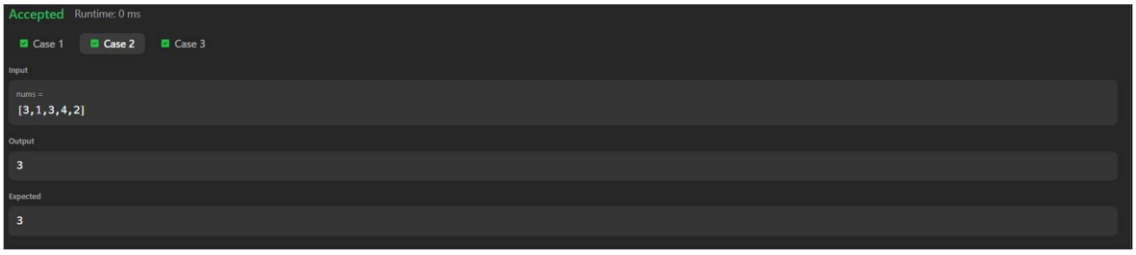
Case 2:

```
Enter no. of vertices: 3
Enter adjacency matrix:
(Use 999 for INF)
0 2 4
2 1 0
4 1 0

Shortest Distance Matrix:
0 2 2
2 1 0
3 1 0
```

Leetcode Problem(287.Duplicate Number):

```
int findDuplicate(int* nums, int numsSize) {
    int slow = nums[0];
    int fast = nums[0];
    while (1) {
        slow = nums[slow];
        fast = nums[nums[fast]];
        if (slow == fast)
            break;
    }
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

OUTPUT:



Lab program 8:(a)Prim's Algorithm

```
#include <stdio.h>
#define INF 999
void prim(int n, int cost[10][10]) {
   int visited[10] = {0};
   int i, j, min, u = 0, v = 0;
   int ne = 1;
   int mincost = 0;
   visited[0] = 1;
   printf("Edges in Minimum Spanning Tree:\n");
   while(ne < n) {
      min = INF;
      for(i = 0; i < n; i++) {
         if(visited[i]) {
            for(j = 0; j < n; j++) {
               if(!visited[j] && cost[i][j] < min) {
                  min = cost[i][j];
                  u = i;
                  v = j;
               }
            }
         }
      }
      printf("%d -> %d = %d\n", u, v, min);
      mincost += min;
      visited[v] = 1;
      ne++;
   }
```

```
    printf("Minimum cost = %d\n", mincost);
}
int main() {
    int n, i, j;
    int cost[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    prim(n, cost);
    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 6
Enter adjacency matrix:
999 1 999 5 4 999
1 999 2 999 6 7
999 2 999 3 999 9
5 999 3 999 999 999
4 6 999 999 999 8
999 7 9 999 8 999
Edges in Minimum Spanning Tree:
Edge      Weight
0 - 1     1
1 - 2     2
2 - 3     3
0 - 4     4
1 - 5     7

Minimum Cost = 17
```

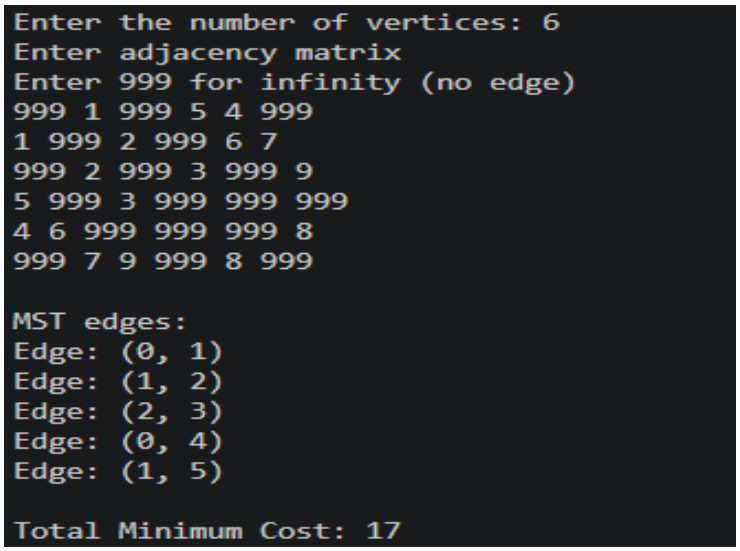
Lab program 8:(b)Kruskal's Algorithm

```
#include <stdio.h>
#define INF 999
int parent[10];
int find(int i) {
   while(parent[i])
      i = parent[i];
   return i;
}
int uni(int i, int j) {
   if(i != j) {
      parent[j] = i;
      return 1;
   }
   return 0;
}
void kruskal(int n, int cost[10][10]) {
   int i, j, min, a, b, u, v;
   int ne = 1;
   int mincost = 0;
   printf("Edges in Minimum Spanning Tree:\n");
   while(ne < n) {
      min = INF;
      for(i = 0; i < n; i++) {
         for(j = 0; j < n; j++) {
            if(cost[i][j] < min) {
               min = cost[i][j];
               a = u = i;
               b = v = j;
```

```
            }
        }
    }
    u = find(u);
    v = find(v);
    if(uni(u, v)) {
        printf("%d -> %d = %d\n", a, b, min);
        mincost += min;
        ne++;
    }
    cost[a][b] = cost[b][a] = INF;
  }
  printf("Minimum cost = %d\n", mincost);
}
int main() {
  int n, i, j;
  int cost[10][10];
  printf("Enter number of vertices: ");
  scanf("%d", &n);
  printf("Enter cost adjacency matrix:\n");
  for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
      scanf("%d", &cost[i][j]);
      if(cost[i][j] == 0)
        cost[i][j] = INF;
    }
  }
  kruskal(n, cost);
```

```
    return 0;
}
```

OUTPUT:



Lab program 9:Fractional Knapsack Using Greedy Technique

```
#include <stdio.h>
struct Item {
    int weight;
    int profit;
    float ratio;
};
void sort(struct Item item[], int n) {
    int i, j;
    struct Item temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(item[j].ratio < item[j + 1].ratio) {
                temp = item[j];
                item[j] = item[j + 1];
                item[j + 1] = temp;
            }
        }
    }
}
void fractionalKnapsack(int n, struct Item item[], int capacity) {
    int i;
    float totalProfit = 0.0;
    sort(item, n);
    for(i = 0; i < n; i++) {
        if(capacity >= item[i].weight) {
            capacity -= item[i].weight;
            totalProfit += item[i].profit;
        }
```

```
        else {
            totalProfit += item[i].ratio * capacity;
            break;
        }
    }
    printf("Maximum profit = %.2f\n", totalProfit);
}
int main() {
    int n, i, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    printf("Enter weights and profits:\n");
    for(i = 0; i < n; i++) {
        scanf("%d%d", &item[i].weight, &item[i].profit);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    fractionalKnapsack(n, item, capacity);
    return 0;
}
```

OUTPUT:

```
Enter number of items: 3
Enter profits:
56 34 78
Enter weights:
24 56 89
Enter knapsack capacity: 43

Selected items (fraction taken):
Item 1 -> 1.00
Item 2 -> 0.21

Maximum Profit = 72.65
```

Lab program 10:Dijkstra's Algorithm

```
#include <stdio.h>
#define INF 999
void dijkstra(int n, int cost[10][10], int source) {
   int distance[10], visited[10];
   int i, j, min, next;
   for(i = 0; i < n; i++) {
      distance[i] = cost[source][i];
      visited[i] = 0;
   }
   distance[source] = 0;
   visited[source] = 1;
   for(i = 1; i < n; i++) {
      min = INF;
      for(j = 0; j < n; j++) {
         if(!visited[j] && distance[j] < min) {
            min = distance[j];
            next = j;
         }
      }
      visited[next] = 1;
      for(j = 0; j < n; j++) {
         if(!visited[j] && min + cost[next][j] < distance[j])
            distance[j] = min + cost[next][j];
      }
   }
   printf("Shortest distances from vertex %d:\n", source);
   for(i = 0; i < n; i++)
      printf("%d -> %d = %d\n", source, i, distance[i]);
```

```
}
int main() {
    int n, i, j, source;
    int cost[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &source);
    dijkstra(n, cost, source);
    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0  20 0 60
100 0 10 60 0
Enter source vertex: 0

Vertex  Distance from Source
0       0
1       10
2       50
3       30
4       60
```

Lab program 11:N-Queens Problem Using Backtracking

```
#include <stdio.h>
#include <stdlib.h>
int x[20];
int count = 0;
int place(int k, int i) {
   int j;
   for(j = 1; j < k; j++) {
      if(x[j] == i || abs(x[j] - i) == abs(j - k))
         return 0;
   }
   return 1;
}
void nQueens(int k, int n) {
   int i, j;
   for(i = 1; i <= n; i++) {
      if(place(k, i)) {
         x[k] = i;
         if(k == n) {
            count++;
            printf("\nSolution %d:\n", count);
            for(j = 1; j <= n; j++) {
               for(i = 1; i <= n; i++) {
                  if(x[j] == i)
                     printf("Q ");
                  else
                     printf(". ");
               }
               printf("\n");
```

```
            }
        }
        else {
            nQueens(k + 1, n);
        }
    }
  }
}
int main() {
  int n;
  printf("Enter number of queens: ");
  scanf("%d", &n);
  nQueens(1, n);
  if(count == 0)
     printf("No solution exists");
  return 0;
}
```

OUTPUT: